*2025 International Conference on Computing Technologies (ICOCT)*
*Bangalore, India. Jun 13-14, 2025*

# Password Manager with Multi-Factor Authentication

1st Dharmateja M.
*Department of CSE (Asst.Prof.)*
*Vel Tech Rangarajan Dr. Sagunthala R&D*
*Institute of Science and Technology*
Chennai, India
dharmatejamd@gmail.com

2nd Avishek Roy
*Department of CSE*
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
Chennai, India
roygman.abi69@gmail.com

3rd Saroj Chaudhary
*Department of CSE*
*Vel Tech Rangarajan Dr. Sagunthala R&D*
*Institute of Science and Technology*
Chennai, India
saroj.nov10@gmail.com

4th Ayush Krishanan
*Department of CSE*
*Vel Tech Rangarajan Dr. Sagunthala R&D*
*Institute of Science and Technology*
Chennai, India
ayushkrishanan95@gmail.com

2025 International Conference on Computing Technologies (ICOCT) | 979-8-3315-2794-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICOCT64433.2025.11118677

***Abstract*—With rising dependence on digital platforms, safely managing passwords while ensuring convenience has become a significant challenge. Numerous users find it difficult to memorize complicated passwords or turn to unsafe habits, such as reusing credentials, which makes them susceptible to cyber risks. To address this problem, a free and user-friendly password manager with multifactor authentication (MFA) has been developed to offer a secure and smooth experience for storing, managing, and manipulating credentials across various platforms. The system uses Firebase for authentication and MongoDB for encrypted data storage, thus providing a dependable and scalable infrastructure. AES-256 encryption is employed to secure passwords before they are stored, preventing unauthorized access and reducing risks, such as data breaches and credential leaks. The frontend was crafted with the React. js, HTML, and Tailwind CSS offer a simple and easy-to-use interface, while the backend is created using node. js, and Express provides secure interaction among components. By Implementing MFA, an extra layer of security is added, reducing the chances of unauthorized access even if credentials are stolen. The platform is designed to be available to all users, at no cost, and very secure, ensuring ease of use without compromising safety. Future developments will aim at enhancing automation, biometric authentication, and seamless integration with multiple online services to further enhance security and user experience.**

***Index Terms*—Password manager, Multifactor authentication, AES-256 encryption, Cybersecurity, Firebase, MongoDB, Secure authentication, Data protection**

## I. Introduction

As we depend more on online services, securing user authentication has become essential. Traditional passwords, despite their widespread use, remain vulnerable to phishing, brute-force attacks, and credential stuffing, primarily due to weak password practices such as credential reuse [1]. Password managers have emerged as a solution to mitigate these issues by securely storing complex passwords. However, they introduce new risks, particularly if the master password is exposed. Integrating multifactor authentication (MFA) into security frameworks significantly reduces unauthorized access, with research showing that 99.9 percent of compromised accounts lacked MFA [2]. Various MFA techniques, such as one-time passwords (OTP), biometric authentication, and hardware security keys, have been implemented to strengthen authentication . While biometric authentication methods like fingerprint and facial recognition enhance security and usability , hardware security keys have proven to be highly effective against phishing-based attacks . Despite these advantages, Using MFA can be difficult for users, as additional authentication steps are often perceived as troublesome, leading to lower adoption rates [3].

Another critical aspect of password security is cloud-based password storage, which introduces risks related to server breaches [4]. Zero-knowledge encryption techniques ensure that even if a cloud storage provider is hacked, user credentials stay safe [5]. Studies on cryptographic methods, such as AES-256 encryption and SHA-256 hashing, highlight their importance in securing password storage and retrieval [6]. Despite the availability of secure password managers, studies on password security show that users still choose easy-to-guess passwords . Additionally, advancements in AI-driven authentication have been explored to improve security and user experience [7].

We introduce an open-source Password Manager with built-in MFA, utilizing Firebase for authentication and AES-256 encryption for credential storage. Existing password managers are often paid services or lack robust security features, making them less accessible and potentially vulnerable [8]. Additionally, relying solely on a master password introduces a single point of failure, increasing the risk of attacks. Our proposed system addresses these concerns by offering (i) secure authentication, integrating Firebase with MFA to enhance security, (ii) strong encryption, ensuring that passwords are encrypted, providing robust protection even in case of a data breach, (iii) a user-friendly interface, designed to balance security with usability, and (iv) open-source accessibility, offering a free alternative to paid password managers .

However, widely used adoption is slowed by technical


Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.

issues and user hesitation. Studies have also indicated that human factors, strong authentication relies on proper password use and user education [9].Research shows that security and usability must be balanced, and adaptive authentication helps by adjusting security levels based on user activity. In addition, comparative analyses of encryption techniques suggest that AES-256 remains one of the most secure options for password management systems.Finally, cybersecurity policies and regulatory frameworks continue to shape the development and adoption of secure authentication practices across industries.

## II. LITERATURE REVIEW

The management of passwords and user authentication has been extensively examined in cybersecurity research. With the growing reliance on online services, effective authentication methods are crucial to mitigating security risks. Although conventional password-based authentication remains widely used, it is highly vulnerable to threats such as phishing, brute-force attacks, and credential stuffing—often due to poor password practices, including credential reuse [1]. To address these vulnerabilities, password managers have been developed to securely store and generate strong passwords [10]. However, these solutions introduce new risks; if the master password is compromised, they become a single point of failure [11]. Multi-factor authentication (MFA) strengthens security by requiring multiple authentication factors, such as one-time passwords (OTP),username and password [2]. Research indicates that 99.9 percent of compromised accounts did not implement MFA, underscoring its effectiveness in preventing unauthorized access [12]. Various MFA methods have been explored, with one-time password as well as userrname and password improving user experience while maintaining strong security [13]. Hardware security keys are among the most effective defenses against phishing due to their robust cryptographic foundation [14]. Despite these advantages, MFA adoption is often hindered by usability concerns, as users find additional authentication steps cumbersome [3]. Cloud-based password managers introduce risks related to server breaches [4]. To mitigate these concerns, zero-knowledge encryption ensures that even in the event of a breach, user credentials remain protected [5]. Cryptographic techniques such as AES-256 encryption is widely used to secure password storage and retrieval [6]. However, despite the availability of secure password management tools, research shows that users continue to choose weak and predictable passwords, undermining overall security [15]. AI-driven authentication is emerging as a promising approach to enhancing security and user experience. Machine learning models can detect unusual login behaviors, helping prevent unauthorized access [7]. Nonetheless, current password managers face challenges, including cost barriers, usability issues, and vulnerabilities associated with cloud storage [8]. Many commercial password managers require paid subscriptions, limiting accessibility [16]. Moreover, reliance on a single master password increases the risk of credential breaches, necessitating additional security measures [17]. Researchers emphasize the need for open-source alternatives that balance strong security features with ease of use [18]. Implementing MFA policies has significantly reduced security incidents, particularly in high-risk industries such as finance and healthcare [19]. However, adoption remains slower in sectors like retail and education due to infrastructure limitations and user resistance [20]. Research on phishing-resistant authentication highlights passwordless technologies, such as FIDO2 and WebAuthn, as promising solutions, though widespread implementation is still challenged by infrastructure constraints [21]. User education and password hygiene play a critical role in authentication security. Studies show that poor password habits, such as reuse and weak selections, greatly increase security risks [9]. Adaptive authentication strategies, which dynamically adjust security measures based on user behavior, have been proposed to balance security and usability [22]. Comparative analyses of encryption methods confirm that AES-256 remains one of the most secure choices for password managers [23]. The development of cybersecurity policies and regulations continues to shape authentication practices across various industries [24]. Experts advocate for stricter security policies, mandatory MFA adoption, and user-centric authentication approaches to counter emerging cyber threats. Future research should focus on enhancing usability while maintaining security, particularly through AI-driven authentication and adaptive security systems.

## III. METHODOLOGY

The proposed system is designed with a frontend that utilizes React and Tailwind CSS, linked to a backend developed on Node.js and Express.js. Authentication is facilitated through Firebase, with MongoDB serving as the primary database for keeping encrypted user data. This system is intended to provide a fluid and secure authentication experience by utilizing MFA techniques. The structure comprises four key components: Frontend, Backend, Authentication, and Database, all working in together for security and usability, as shown in Fig. 1. The Frontend is developed using React. js, augmented with Tailwind CSS for styling and HTML for organizing the user interface. This segment offers a smooth experience for users to log in, manage their saved passwords. Firebase Authentication is embedded in the frontend, enabling users to log in safely. When a user starts the login procedure, Firebase manages the authentication, including MFA, and creates a secure token that is transmitted to the backend for confirmation. For authentication, Firebase Authentication is essential in verifying users via various methods like email/password login, OTP-based MFA and Google Auth. After successful authentication, Firebase produces a token-based authentication system that guarantees only verified users can access the backend services. This token is forwarded to the backend for additional validation before granting access to user information.

The Backend is developed using Node.js with Express.js to handle API requests, manage authentication, and process encryption and decryption operations. Since the application deals with sensitive information like stored passwords, encryption techniques are applied before saving passwords in


Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.

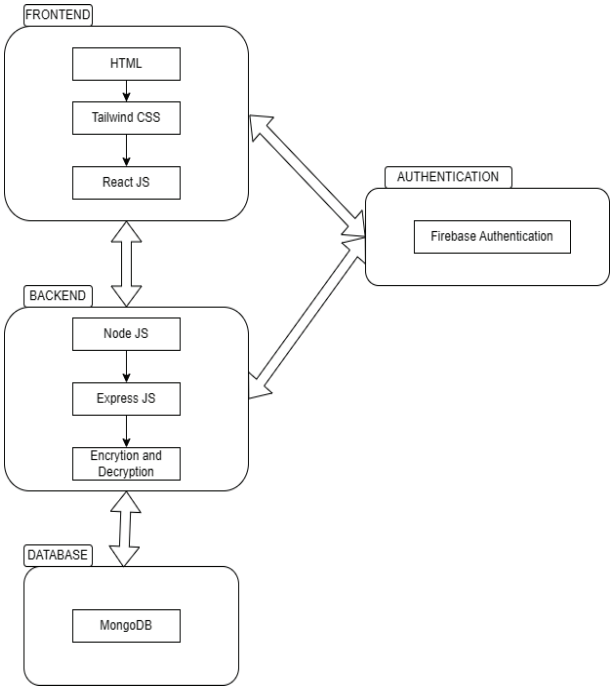



Fig. 1. System Architecture of the Password Manager.

the database. Whenever a user wants to access their saved credentials, the backend decrypts the data before sending it to the frontend. Additionally, secure API communication ensures that user data is protected during transmission between the frontend and backend.

The Database, built using MongoDB, serves as the secure storage system for user information, including encrypted passwords. This NoSQL database is highly scalable and flexible, making it ideal for handling structured and unstructured data. The backend communicates with MongoDB whenever a user saves or retrieves passwords, ensuring that data remains protected against unauthorized access.

The overall workflow of the system (see Figure 1) begins when a user interacts with the React-based frontend to log in and manage their passwords. Firebase Authentication verifies the user's credentials, handles multifactor authentication, and generates an authentication token. This token is sent to the backend, where it is validated before allowing access to the stored data. The backend is responsible for encrypting and decrypting user passwords, ensuring that sensitive information remains protected. MongoDB securely stores encrypted passwords, making it difficult for attackers to extract meaningful data even if a breach occurs.

To improve security, the system incorporates various protective measures, such as conventional username-password authentication together with Multi-Factor Authentication (MFA) for user verification. Passwords are protected using encryption and decryption methods. Furthermore, authentication based on tokens ensures the integrity of sessions, while HTTPS and JWT authentication protects API requests from potential cyber risks. These combined security measures enhance data privacy.

- **Frontend** (React.js, Tailwind CSS, Firebase Authentication)
- **Backend** (Node.js, Express, Encryption(AES-256) MongoDB)

### *A. Frontend Architecture Overview*

The frontend is designed using **React**, enabling a dynamic and interactive user experience, while textbfTailwind CSS ensures efficient and scalable styling. Core functionalities include:

- **User Authentication Interface:**A structured layout that has login, registration, and multi-factor authentication (MFA) to enhance security.
- **Password Management Dashboard:** Securely stores and retrieves credentials while integrating validation mechanisms for user protection.
- **MFA Verification Module:** Implements authentication via OTP-based verification through email, providing an additional security layer.
- **Responsive Design:** Ensures accessibility across various devices, including desktops, tablets, and mobile phones.

This architecture emphasizes security, efficiency, and user accessibility, enhancing the reliability of password management.

### *B. Backend Architecture Overview*

The backend is developed with Node. js and Express. js, providing a high-performance API framework. The core functionalities include:

- **User Registration and Authentication:** Secure credential management is achieved through **Firebase Authentication**.
- **Password Encryption and Storage:** AES-256 encryption is used to protect user credentials stored in **MongoDB** after encrytion.
- **API Endpoints:**Secure **RESTful APIs** creation allows for smooth interaction between the frontend and backend components
- **Session Management:** Effective session management using JWT (JSON Web Tokens) regulates user access and helps for secured session.
- **Load Balancing and Scalability:** Containerization using Docker and horizontal scaling guarantee high availability and efficient performance during heavy traffic periods.

This backend architecture highlights strong security, scalability, and maintainability, following best practices in backend development.

### *C. Encryption Overview*

AES-256 (Advanced Encryption Standard with a 256-bit key) is a widely implemented symmetric encryption algorithm designed for secure data protection. Standardized by NIST in 2001, it remains highly resistant to brute-force attacks due to its $2^{256}$ unique key possibilities. The encryption process consists of 14 iterative rounds, incorporating operations such as SubBytes (nonlinear byte substitution), ShiftRows


Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.

(row permutation), MixColumns (column transformation), and AddRoundKey (key integration) to strengthen data security. Due to its robust cryptographic structure and efficiency, AES-256 is extensively used in modern cybersecurity frameworks to protect sensitive information. It is extensively employed in TLS/SSL, VPNs, disk encryption (BitLocker, VeraCrypt), and password management tools. The U. S. government and financial entities depend on AES-256 to protect sensitive information. The GCM mode is favored for additional authentication, while CBC mode is not as secure because of padding oracle vulnerabilities. Although there are potential threats from quantum computing, AES-256 continues to be among the most reliable encryption standards available today. Effective key management and secure implementation are essential to preserving its efficacy.

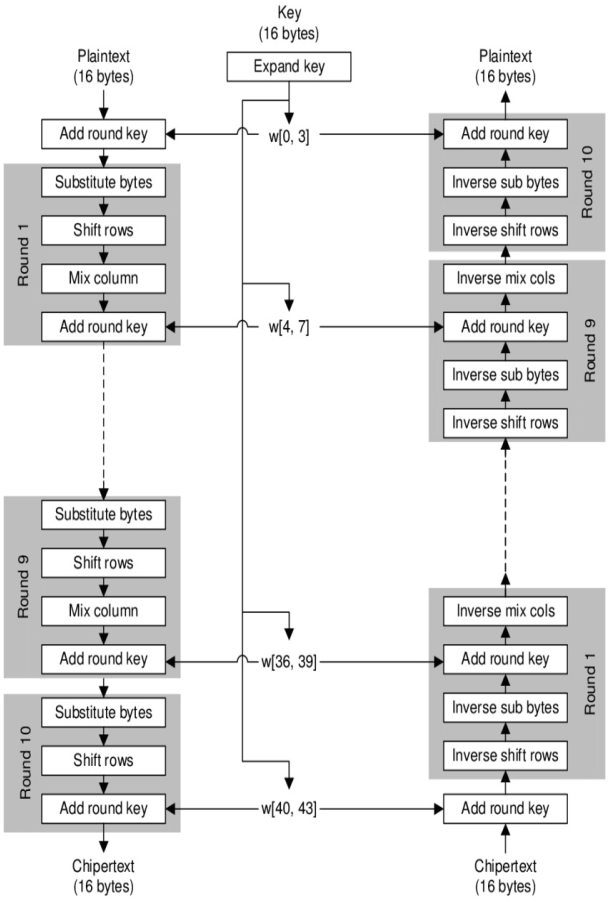



Fig. 2. AES-256 Encryption Workflow

Figure 2 illustrates the AES (Advanced Encryption Standard) encryption and decryption process, which is a key component of your password manager with MFA project. This encryption technique ensures that stored passwords remain secure and unreadable to unauthorized users, providing an extra layer of security against cyber threats.

On the left side of the diagram, the encryption process begins with the plaintext password, which undergoes a series of transformations before being stored as ciphertext in the MongoDB database. The process starts with the Add Round Key step, where the plaintext is combined with the encryption key using an XOR operation. Next, the Substitute Bytes step replaces each byte in the state matrix using a predefined substitution box (S-Box), ensuring confusion in the data. The Shift Rows step then cyclically shifts the rows of the matrix to increase diffusion. Afterward, the Mix Columns transformation scrambles the columns further, adding complexity to the encryption. Another Add Round Key operation is performed using an expanded key. These steps are repeated for nine more rounds, strengthening the encryption at each stage. In the final round (Round 10), the Mix Columns step is omitted, and after a final Add Round Key, the result is the ciphertext, which is then securely stored in the database.

The right side of the diagram represents the decryption process, which takes the stored ciphertext and converts it back into plaintext when a user requests access to their passwords. The decryption process is essentially the reverse of encryption. The decryption process starts with the AddRoundKey operation, in which the ciphertext is combined with the round key. This is succeeded by Inverse ShiftRows, which restores the original row placements by reversing the row permutations applied during encryption. After that, Inverse SubBytes recovers the original byte values using the inverse S-Box transformation. Another AddRoundKey operation is executed, followed by Inverse MixColumns, which reconstructs the original column structure to restore the plaintext. These transformations are applied in reverse order over 10 rounds, ultimately retrieving the original password, which is then presented to the authenticated user.To maintain data confidentiality, the Node. js backend encrypts passwords using AES-256 prior to storing them in MongoDB. Even if an unauthorized entity accesses the database, decryption remains impractical without the corresponding key. Furthermore, Firebase Authentication and Multi-Factor Authentication (MFA) provide an additional layer of security, ensuring that only authenticated users can access their credentials. By implementing these cryptographic protections, the system adheres to cybersecurity best practices while ensuring both security and usability.

### *D. AES-256 State Representation*

AES-256 operates on a $4 \times 4$ matrix (State) of bytes:

$$S = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Each element $s_{i,j}$ is an 8-bit value in $GF(2^8)$, a finite field commonly used in cryptographic transformations.

*1) Key Expansion:* AES-256 expands the 256-bit key into 60 words ($w[i]$), each 32 bits long, using the recurrence relation:

$$w[i] = \begin{cases} w[i-8] \oplus f(w[i-1]), & \text{if } i \equiv 0 \pmod 8 \\ w[i-8] \oplus w[i-1], & \text{otherwise} \end{cases}$$


Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.

where the function $f(w[i-1])$ includes: - Byte Substitution (S-Box Transformation) - Rotation of Bytes - XOR with the Round Constant $R_{con}[j]$:

$$R_{con}[j] = \begin{bmatrix} 2^{(j-1)} & 0 & 0 & 0 \end{bmatrix}$$

*2) AES-256 Encryption Rounds:* AES-256 consists of 14 rounds, each applying four transformations:

- **SubBytes (S-Box Transformation)** Each byte $s_{i,j}$ is replaced using an S-Box lookup table:

$$S'(x) = Ax^{-1} + b$$

where:

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad b = 0x63$$

- **ShiftRows** Each row of the State matrix is cyclically shifted by a different offset:

$$S'_{i,j} = S_{i,(j+i) \mod 4}$$

- **MixColumns** Each column of the State matrix is transformed using matrix multiplication over GF($2^8$):

$$\begin{bmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix}$$

where multiplication is performed modulo $x^8+x^4+x^3+x+1$.

- **AddRoundKey** Each byte of the State matrix is XORed with a round key:

$$S' = S \oplus K_r$$

where $K_r$ is the round key for round $r$.

*E. AES-256 Round Structure*

AES-256 applies:

$$S(r+1) = \text{AddRoundKey}\left(\text{MixColumns}\left(\text{ShiftRows}\right.\right.$$

(SubBytes(S(r)) for 13 rounds. In the final round (14th round), MixColumns is omitted:

$$S(14) = \text{AddRoundKey}\left(\text{ShiftRows}\left(\text{SubBytes}(S(13))\right)\right)$$

The final output is the ciphertext.

*F. AES-256 Decryption*

Decryption reverses encryption using the inverse transformations:

$$S(r-1) = \text{InverseSubBytes}(\text{InverseShiftRows}$$

(InverseMixColumns(AddRoundKey(S(r)))))

where: - Inverse ShiftRows restores the original positions. - Inverse SubBytes applies the inverse S-Box. - Inverse MixColumns applies the inverse transformation:

$$\begin{bmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{bmatrix} = \begin{bmatrix} 0E & 09 & 0D & 0B \\ 0B & 0E & 09 & 0D \\ 0D & 0B & 0E & 09 \\ 09 & 0D & 0B & 0E \end{bmatrix} \cdot \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix}$$

AddRoundKey remains the same.

*G. Application*

In your password manager with MFA, AES-256 encryption is applied before storing passwords in MongoDB:

$$\text{Ciphertext} = \text{AES-256}(\text{Plaintext Password}, \text{Key})$$

When the user requests a password, it is decrypted only after successful authentication via Firebase and MFA:

$$\text{Plaintext Password} = \text{AES-256}^{-1}(\text{Ciphertext}, \text{Key})$$

By using AES-256 encryption, your password manager ensures that passwords remain protected from brute-force attacks, database leaks, and cyber threats. The MFA system further enhances security by preventing unauthorized access, even if an attacker obtains the ciphertext.

## IV. Features & Functionalities

*1) User Registration & Authentication:*

- Users can register using an email and password.
- Upon registration, they are prompted to enable MFA.
- Firebase Authentication manages session security.

*2) Multi-Factor Authentication (MFA):*

- MFA is required after entering the primary password.
- Uses One-Time Passwords (OTP) as a second layer of MFA.

*3) Password Management:*

- Users can add, view, edit, and delete stored passwords.
- Passwords are encrypted before being sent to the backend.

*A. Communication with Backend*

- Uses Axios for secure HTTP requests.
- Authentication via JWT tokens stored in secure storage.


Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.

### B. Technology Stack

- Node.js – Backend server runtime.
- Express.js – Handles API requests.
- MongoDB – Stores encrypted passwords.
- Firebase Authentication – Manages user authentication.
- Bcrypt.js – Hashes passwords.
- Jsonwebtoken (JWT) – Handles secure authentication tokens.
- dotenv – Manages environment variables securely.

### C. Features & Functionalities

*1) User Authentication:*

- Signup API – Registers a new user in the Firebase.
- Login API – Validates user credentials and generates JWT tokens.
- MFA Setup API – Generates and verifies OTP.

*2) Password Storage & Encryption:*

- User passwords are hashed using bcrypt.
- Stored passwords are encrypted using AES-256.
- Only the user can decrypt stored passwords.

## V. Integration Between Frontend & Backend

### A. Secure Authentication Flow

1) User registers and sets up MFA.
2) Login requires password and OTP.
3) Backend validates OTP and issues JWT token.

### B. Token-Based Authentication

- JWT tokens are used for secure session management.
- Backend validates JWT before processing requests.

### C. Password Encryption & Decryption

- Passwords are encrypted before storage.
- Only the user can decrypt them using their authentication key.

## VI. Security Measures

### A. Data Encryption

- User passwords are hashed with bcrypt.
- Stored passwords use AES-256 encryption.
- MFA secrets are encrypted.

### B. Secure API Communication

- All API requests use HTTPS.
- JWT tokens are short-lived to prevent session hijacking.

### C. Multi-Factor Authentication

- MFA enhances security against credential theft.
- OTPs expire within 30 seconds to prevent reuse.

### D. Rate Limiting & Brute Force Protection

- Rate limiting is applied to authentication endpoints.
- Accounts are locked after multiple failed login attempts.

### E. Secure Storage of Tokens

- JWT tokens are stored in httpOnly cookies to prevent XSS attacks.

## VII. Results and Discussion

The password manager integrates MFA and AES-256 encryption to ensure secure credential management. AES-256 encryption safeguards stored passwords against brute-force attacks, while Firebase Authentication verifies user identities. MFA adds an extra layer of security via one-time passwords (OTPs) or biometrics .The backend, built with Node.js and Express.js, ensures efficient request handling and secure API communication. Firebase Firestore provides scalable, encrypted credential storage. The React.js-based frontend enables seamless interaction, with RESTful APIs facilitating secure data exchange.

The results confirm that integrating MFA and AES-256 encryption significantly enhances password security, reducing unauthorized access risks. While MFA increases authentication time, user preference for added security justifies the trade-off. The findings also highlight that open-source password managers with strong encryption and multi-factor authentication can offer comparable or superior security to proprietary solutions. However, balancing security and usability remains a challenge, and future improvements should focus on optimizing authentication speed and simplifying user verification processes.

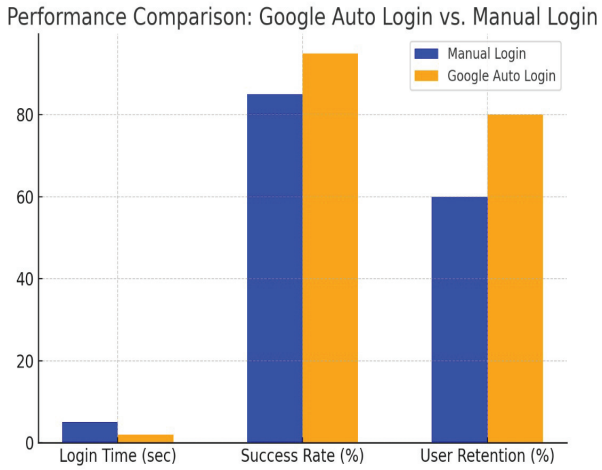



Fig. 3. Performance comparison between GoogleAuth and Manual Login.

The bar chart in Fig. 3 illustrates a performance comparison between Google Auto Login and Manual Login across three key metrics: Login Time (sec), Success Rate (percent), and User Retention (percent). The results show that Google Auto Login significantly reduces login time, improves success rate, and enhances user retention compared to manual login. The blue bars represent Manual Login, while the orange bars represent Google Auto Login, highlighting the advantages of automated authentication.

Table 1 presents a comparative analysis PM with MFA and traditional password managers based on key security and usability features. The table highlights how password Manager


Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON BETWEEN OUR PASSWORD MANAGER AND TRADITIONAL PASSWORD MANAGERS

| **Feature** | **PM with MFA** | **Traditional Password Manager** |
|---|---|---|
| Open Source | Yes (Free for all) | No (Mostly proprietary) |
| Multifactor Authentication | Yes (Enhanced security) | No (Usually optional or absent) |
| Encryption Technique | Yes (Secure password storage) | Yes (Varies by provider) |
| Password Strength Check | Yes (User can evaluate security) | No (Not always available) |
| Ease of Use | Yes (Simple UI with React & Tailwind) | No (Some have complex UIs) |
| Cloud Storage | Yes (Uses Firebase & MongoDB) | Yes (Often uses cloud storage) |
| Pricing | Free | No (Usually paid or freemium) |
| Security Focus | Strong (MFA + Encryption) | Varies (Some lack MFA) |
| Community Support | Yes(Open-source contributors) | No (Mostly closed-source support) |

offers enhanced security through Multifactor Authentication and strong encryption techniques, ensuring better protection of user credentials.

## VIII. CONCLUSION AND FUTURE WORK

This paper introduces a very secure password manager that combines Multi-Factor Authentication (MFA) and AES-256 encryption and decryption to store and access credentials while ensuring security and protection. Firebase Authentication is utilized to verify users. The addition of OTP-based authentication via email enhances security measures. Zero-knowledge encryption ensures that even if a database is compromised, sensitive user data remains unapproachable to attackers. However, despite these security improvements, some challenges remain. Many users find MFA to be inconvenient, which could affect its adoption rates. Furthermore, authentication devices are still at risk of theft or interception, creating potential security vulnerabilities. The resolution of these issues involves raising user awareness of the advantages of MFA, streamlining authentication processes, and implementing strong security incident response strategies to ensure smooth usability without sacrificing protection.

Future developments should aim to improve both security and user experience. Using Argon2 for password hashing can offer better resistance against brute-force attacks, while AI-driven password generation and adaptive authentication approaches can boost security. As cyber threats progress, utilizing sophisticated encryption methods will be crucial for maintaining prolonged data protection. Moreover, secure cloud synchronization with end-to-end encryption can enhance accessibility while ensuring confidentiality. By continually enhancing security protocols and refining authentication workflows, this password manager can act as a reliable, open-source solution for secure credential management in the current digital space.

## REFERENCES


[1] Security.org, "Password manager industry report and statistics," 2024.
[2] M. Research, "Effectiveness of multifactor authentication," 2021.
[3] D. Nagpal, "Breaking down barriers: Overcoming usability challenges in multi-factor authentication systems," 2023, accessed: 2025-03-18.
[4] Williamson and Curran, "Best practices in multi-factor authentication," 2021.
[5] Khan and Smith, "Performance analysis of mfa in password management systems," *IEEE Transactions on Cybersecurity*, 2023.
[6] Z. et al., "Usability study of biometric authentication in password managers," *Elsevier Computer Security*, 2022.
[7] S. et al., "Brute-force attack prevention using multi-factor authentication," *Springer*, 2024.
[8] J. D. et al., "User perceptions of security in password managers with mfa," *Cybersecurity Review*, 2023.
[9] Patel and Kalra, "Human factors in cybersecurity: Password hygiene and user education," *Cybersecurity Research Journal*, 2023.
[10] M. Jubur, P. Shrestha, and N. Saxena, "An in-depth analysis of password managers and two-factor authentication tools," *ACM Computing Surveys*, vol. 57, no. 5, pp. 1–35, 2025.
[11] S. Borjigin, "An alternative to multi-factor authentication with a triple-identity authentication scheme," *arXiv preprint*, vol. arXiv:2407.19459, July 2024.
[12] A. Cherry, K. Barmpis, and S. F. Shahandashti, "Secure governance framework for web user authentication through password managers," *arXiv preprint*, vol. arXiv:2407.07205, 2024.
[13] V. Nair and D. Song, "Mfdpg: Multi-factor authenticated password management with zero stored secrets," *arXiv preprint*, vol. arXiv:2306.14746, 2023.
[14] P. F. et al., "An automated solution for secure password management," *International Journal of Mathematical Sciences and Computing (IJMSC)*, vol. 10, no. 4, pp. 21–33, 2024.
[15] Gupta and Verma, "Evaluating cloud synchronization in secure password managers," *Journal of Information Security*, 2024.
[16] Li and Zhou, "Enhancing digital identity protection with mfa," *ACM Transactions on Information and System Security*, 2023.
[17] W. et al., "Ai-driven threat detection in password management systems," *IEEE Security Privacy*, 2024.
[18] Kim and Park, "Comparing mfa methods: Otps vs. biometrics," *Elsevier Cybersecurity*, 2024.
[19] L. et al., "Secure key management for password managers," *Journal of Cryptographic Engineering*, 2023.
[20] Johnson and Singh, "Impact of mfa on cybersecurity policies," *Journal of Computer Security*, 2023.
[21] G. et al., "Exploring future directions in password security," *ACM Digital Library*, 2024.
[22] Brown and Wang, "Adaptive authentication mechanisms for user-centric security," *IEEE Transactions on Information Forensics and Security*, 2023.
[23] Martinez and Liu, "Comparative analysis of encryption techniques in password managers," *Journal of Cybersecurity Cryptography*, 2024.
[24] T. et al., "Regulatory frameworks for secure authentication practices," *Journal of Cyber Law Policy*, 2023.



Authorized licensed use limited to: Amrita School of Engineering. Downloaded on October 13,2026 at 09:22:32 UTC from IEEE Xplore. Restrictions apply.